

Supplementary Information

Solutions to the challenges proposed by 0xPARC

Alberto Hernandez-Espinosa

17 September 2026

Contents

S1	Questions, conventions, and contribution	3
S2	Q2: majority using only three-input majority	3
S3	Index deconvolution: representation, recovery and controls	4
S3.1	Exact index sets and schema expansion	4
S3.2	Isolation of functionally responsive coordinates	4
S3.3	Majority checks and negative controls	5
S3.4	The five-input network and pattern figures	5
S4	Q5 and Q6: range and exclusion constraints	6
S4.1	What was stated, and what the method returned	6
S4.2	From recovered gates to quadratic rows	7
S4.3	Q5: the range predicate	7
S4.4	Q6: exclusion of one	8
S5	Q7: a 64-bit factorisation	8
S5.1	The multiplier built from recovered cells	8
S5.2	The exhaustive ladder and the law it fixes	8
S5.3	The direct system, retained as a cross-check	9
S6	Q8: a 4096-bit factorisation	9
S7	Verification record and reproducibility	11
S7.1	Recovered-cell arithmetic route	11
S7.2	Specified validation inputs	12
S7.3	Why the omitted-bound controls are informative	13
S7.4	Constraint counts	13
S7.5	Reading the role ledger	13
S7.6	Reproduction and scope of the evidence	15
S8	General capacities and developmental scope of index deconvolution	15
S8.1	The capacity map	16
S8.2	Compression and the Kolmogorov perspective	16
S8.3	Prediction, explanation and scale	19
S9	What the majority ladder measures	19
S9.1	Four counts, not one	20

S9.2	Measured values	20
S9.3	Two growth rates, and why they differ	20
S9.4	What the independence of the specification amounts to	21
S9.5	The recovered name is a canonical choice, not a unique one	21
S9.6	Consequence for the comparison in Section S6	21

S1 Questions, conventions, and contribution

We keep the source’s numbering of its eight puzzles [1] and address five of them: majority (Q2), range (Q5), exclusion of one (Q6), the 64-bit factorisation (Q7) and the 4096-bit factorisation (Q8). List recovery (Q1), the Fourier circuit (Q3) and the modular power sums (Q4) are omitted, because index deconvolution does not supply their answers and this document reports only what it does supply. The constraint field in Q5–Q8 is $\mathbb{F}_P$, with

$$P = 21888242871839275222246405745257275088548364400416034343698204186575808495617.$$

External field values must be canonical integers in $[0, P - 1]$.

This supplement states the exact conventions and gives the full arguments behind the main text. The Boolean reconstruction is described in Section S3; the finite verification record is in Section S7.

S2 Q2: majority using only three-input majority

Let f be a monotone Boolean function and choose distinct active variables a, b, c . Write $f_{b \leftarrow a}$ for the function formed by substituting a for b , and similarly for the other identifications. Then

$$f = \text{MAJ}_3(f_{b \leftarrow a}, f_{c \leftarrow b}, f_{a \leftarrow c}).$$

If the three bits agree, every substitution leaves them unchanged. Otherwise one substitution leaves the assignment unchanged, one increases a bit, and one decreases a bit. By monotonicity the three outputs are f , a value at least f , and a value at most f . Their majority is f .

Odd-input majority is monotone and self-dual: complementing all input bits complements its output. Identification preserves both properties. A monotone self-dual Boolean function on at most two active variables is a projection: its values at 00, 11 are 0, 1, and the values at 01, 10 are opposite. Thus recursion terminates in input wires and uses only MAJ_3 gates.

The construction represents subproblems by a positive odd total of weights over the original coordinates. Identification merges weights, a weight greater than half the total immediately returns its input, and otherwise the first three active variables are used. Memoisation identifies equal weight tuples. Every operand is an external input or an earlier gate; ordered duplicate ports and free fan-out are permitted. Constants and inversions are absent.

With $T(2) = 0$ and $T(k) \leq 1 + 3T(k - 1)$, a loose unshared bound is $T(n) \leq (3^{n-2} - 1)/2$ for $n \geq 3$. This proves the requested existence for 2025 inputs but is not a compactness guarantee. No 2025-input circuit is materialised here. Gate counts below refer only to emitted circuits. Exhaustive checks cover odd sizes through 11. Beyond that, enumeration is replaced by identity on a canonical decision diagram, which settles agreement on all 2^n states without visiting any: the emitted circuit is confirmed equal to an independently built threshold recurrence at $n = 13, 15, 21, 31, 41, 51, 63, 101$ and 151 . At $n = 151$ this is 8,116,950 gates and 41,196,123 decision nodes. The threshold target shares no code with the majority builder.

For comparison, a naive majority of disjoint group majorities fails: the groups 110, 110, 000 produce local votes 1, 1, 0 and hence output one, although only four of the nine original inputs are one.

S3 Index deconvolution: representation, recovery and controls

S3.1 Exact index sets and schema expansion

Let $f : \{0, 1\}^n \rightarrow \{0, 1\}$ be a deterministic output column. States are ordered by $\iota(x) = \sum_{i=0}^{n-1} 2^i x_i$, with bit zero varying fastest. The active index set $\mathcal{I}_f$ consists of all indices with output one. Membership in this set specifies the complete output column.

For a schema $\sigma \in \{0, 1, *\}^n$, let $J_1(\sigma)$ be its fixed-one positions and $J_*(\sigma)$ its don't-care positions. Define the decimal anchor and the offset family by

$$a_\sigma = \sum_{i \in J_1(\sigma)} 2^i, \quad \Omega_\sigma = \left\{ \sum_{i \in J_*(\sigma)} 2^i z_i : z_i \in \{0, 1\} \right\}.$$

The summands are the fillings of the schema's own don't-care positions; Ω_σ is the numerical offset family these fillings generate. The indices represented by the schema are exactly $a_\sigma + \Omega_\sigma$. Fixed-one and free positions are disjoint, so adding their encodings cannot create a carry. A covering family $\mathcal{C}$ represents the output precisely when

$$\mathcal{I}_f = \bigcup_{\sigma \in \mathcal{C}} (a_\sigma + \Omega_\sigma).$$

This is a union, since schemata may overlap. One must retain each anchor's associated offset family; arbitrary cross-pairing of anchors and offsets need not preserve the function. A cover by complete assignments always exists, but a compact cover or a minimal cover is not guaranteed.

For three-input majority, $\mathcal{C} = \{11*, 1*1, *11\}$ gives anchor/offset pairs $(3, \{0, 4\})$, $(5, \{0, 2\})$, $(6, \{0, 1\})$. The expansion is $\{3, 5, 6, 7\}$. All three inputs affect the function, although each schema has a free position. Disconnected coordinates are free in every schema; freedom in one schema does not imply disconnection.

S3.2 Isolation of functionally responsive coordinates

From the complete column, recover

$$I_c(f) = \{i : \exists x, f(x) \neq f(x \oplus e_i)\},$$

where $x \oplus e_i$ flips bit i . These are the functional inputs. If $i \notin I_c(f)$, changing only bit i never changes the output. Any two states agreeing on $I_c(f)$ can be joined by a sequence of flips outside $I_c(f)$, so their outputs agree. For a static one-output projection this is an exact consistency identity, but the inverse method uses it to isolate an output condition within the full integrated description; it does not discard the remaining nodes or claim to produce a reduced truth table for the whole system. In a dynamical network, the relevant observation is instead indexed by the selected node union, state and time or intervention condition. Different selections can expose different outputs of the same evolving system. A declared model family may still be used to name an isolated pattern, but multiple names or internal implementations can describe the same behaviour.

Testing all coordinate pairs costs at most $n2^{n-1}$ comparisons per output column, with $O(2^n)$ storage when the full column is retained. The method does not remove the exponential cost of supplying an arbitrary complete repertoire. A compact schema family is a representation benefit, not a universal guarantee that this inverse computation is inexpensive.

For a network with observed outputs for every input state, apply the same argument to each output column to recover a functionally equivalent map. Structurally declared but functionally inactive inputs are not identifiable. If only a circuit's final output is observed, internal gates and their wiring are not identifiable either. Partial or noisy observations do not satisfy this exact-recovery premise.

S3.3 Majority checks and negative controls

For odd n , setting exactly $(n - 1)/2$ of the other bits to one shows that every input of majority is functional. Complementation gives exactly 2^{n-1} active indices. The added manuscript check evaluates the emitted circuits for $n = 1, 3, 5, 7, 9, 11$, applies functional-input detection and column reduction, and compares the result with the independent condition $\sum_i x_i > n/2$ on every state. This covers $2 + 8 + 32 + 128 + 512 + 2048 = 2730$ states. The check also expands the three schemata above exactly.

A free-input control embeds three-input majority in five coordinates. All 32 states recover $I_c = \{0, 1, 2\}$ and the eight-entry majority table; coordinates 3 and 4 do not affect the output. An incorrect-circuit control uses a majority of three disjoint three-input majorities. It has nine functional inputs and 256 active indices, yet its full output column differs from nine-input majority. Thus neither the number of functional inputs nor the size of the active set establishes equality; exact membership is checked. The main text's 110, 110, 000 assignment is one explicit disagreement.

S3.4 The five-input network and pattern figures

The worked network uses the emitted five-input circuit, with

$$\begin{aligned} g_0 &= \text{MAJ}_3(x_0, x_2, x_4), & g_1 &= \text{MAJ}_3(x_0, g_0, x_3), \\ g_2 &= \text{MAJ}_3(x_1, x_3, x_4), & g_3 &= \text{MAJ}_3(x_0, x_1, g_2), \\ g_4 &= \text{MAJ}_3(x_2, x_3, x_4), & g_5 &= \text{MAJ}_3(x_1, x_2, g_4), \\ y &= g_6 = \text{MAJ}_3(g_1, g_3, g_5). \end{aligned}$$

The figures show this circuit's connections, not a generic illustrative network. All seven internal outputs are evaluated for all 32 inputs. The output column is compared state by state with $\sum_i x_i \geq 3$; functional-input detection recovers all five coordinates.

To generate the schema family, choose every three-element subset of the five input positions, fix those bits to one and leave the other two free. This gives $\binom{5}{3} = 10$ schemata. Expanding their associated anchors and offsets and taking the union gives precisely the circuit's 16 active indices. The displayed 111 * * schema has anchor 7 and offsets 0, 8, 16, 24. Its four complete input states, together with their internal outputs, are

Index	Input $x_0 \dots x_4$	Internal outputs $g_0 \dots g_6$
7	11100	1101011
15	11110	1111111
23	11101	1111111
31	11111	1111111

For each input, the figure-generation check retains a pair of states that differ only at that input and have different outputs. The displayed pair is index 6 (01100, output zero) and index 7 (11100, output one). The complete repertoire, internal signals, schemata and checks accompany the figures as numerical data. Both diagram topology and table entries are derived from the same circuit, and every schema is checked for exact membership before rendering. These controls establish the stated finite reconstruction behaviour and illustrate its limitations; they are not a new synthesis algorithm or a claim to recover hidden internal topology.

S4 Q5 and Q6: range and exclusion constraints

S4.1 What was stated, and what the method returned

The cells behind Q5, Q6 and Q7 were obtained by one procedure, described here once and used by all three. For each cell an integer relation is written as a map from an input bit vector to an output bit, with nothing said about how the map is to be computed. The relation is evaluated over its complete input space, the resulting behaviour is turned into a decision-diagram root, and that root is handed to index deconvolution, which reports the coordinates the output actually responds to and matches the restricted behaviour against the canonical gate family of Section S3. The returned gate is what the compiler receives. No gate literal appears in the builders that assemble the comparison scan, the inequality scan or the multiplier; every gate they place is read from a recovery record.

For the range and exclusion questions the stated relations are two scan steps. The comparison step has inputs (ℓ, e, x) , the incoming “less”, the incoming “equal” and the value bit at this position, and a bound bit β which is a constant of the position rather than an input. Its two outputs are stated as

$$\ell_{\text{out}} = \ell \vee (e \wedge \bar{x} \wedge \beta), \quad e_{\text{out}} = e \wedge [x = \beta],$$

which is the meaning of a running comparison and not a circuit for it. The inequality step has inputs (d, x) and a constant bit γ , with the single output $d_{\text{out}} = d \vee [x \neq \gamma]$. Because β and γ are constants of the position, each step is recovered twice, once for each value.

Table 1 is the recovery record as the method produced it. The connected inputs are the coordinates found essential by the cofactor test; the gate is the canonical name and the match class is the full set of families whose exact behaviour agrees, so that the name may be read as the choice within that class which it is.

Table 1: The recovery record for the six arithmetic cells. Inputs are numbered in the order they are stated. The match class is every canonical family that reproduces the recovered behaviour exactly, so a class of size one is a unique name and a larger class is not. The unary cell matches seven families because a plain wire is expressible in nearly all of them.

Cell	Output	Connected	Gate	Match class
full adder	sum	0, 1, 2	XOR	XOR
	carry	0, 1, 2	MAJORITY	KOFN, MAJORITY, REGULATORY_DNF
partial product	product	0, 1	AND	AND, KOFN, MAJORITY, REGULATORY
comparison, $\beta = 1$	less	0, 1, 2	REGULATORY_DNF	REGULATORY_DNF
	equal	1, 2	AND	AND, KOFN, MAJORITY, REGULATORY
comparison, $\beta = 0$	less	0	AND	AND, CANALISING, KOFN, MAJORITY, OR, REGULATORY, XOR
	equal	1, 2	NIMPLIES	CANALISING, NIMPLIES, REGULATORY
inequality, $\gamma = 0$	differs	0, 1	OR	CANALISING, KOFN, OR, REGULATORY_DNF
inequality, $\gamma = 1$	differs	0, 1	REGULATORY_DNF	REGULATORY_DNF

Read as expressions, the record says

$$\begin{aligned}\beta = 1 : \quad \ell_{\text{out}} &= \ell \vee e\bar{x}, & e_{\text{out}} &= e \wedge x, \\ \beta = 0 : \quad \ell_{\text{out}} &= \ell, & e_{\text{out}} &= e \wedge \bar{x}, \\ \gamma = 0 : \quad d_{\text{out}} &= d \vee x, \\ \gamma = 1 : \quad d_{\text{out}} &= d \vee \bar{x}.\end{aligned}$$

The two clauses of the $\beta = 1$ “less” cell are returned as a regulatory disjunctive form with ℓ as a lone activator and e activating under the inhibition of x , which is exactly $\ell \vee e\bar{x}$; the $\gamma = 1$ cell is returned the same way as $d \vee \bar{x}$. At $\beta = 0$ the method reports a single connected input for “less” and names the unary gate, which is the wire itself: two of the three stated dependencies are absent from the behaviour, because below a zero bound bit no new strict inequality can arise. This specialisation is a finding of the cofactor test, not a simplification applied to a circuit afterwards; no circuit exists at that point to simplify.

A recovered gate names a function of whatever arity it found essential, while the lowering carries two- and three-input gates, so the expansion between them is a proof obligation. It is discharged by root identity: the expanded network is evaluated back into a root, the named gate is built independently by the engine’s own gate builder, and the two roots are compared. All nine expansions in use are checked this way, together with every candidate the identifier can return at arities one to five and every function of one to three variables.

S4.2 From recovered gates to quadratic rows

A quadratic row means $A(w)B(w) = C(w)$ in $\mathbb{F}_P$, with all three expressions linear in the signals. Both relations are compiled from the cells above, and the two field-level equations they carry are consequences of putting Boolean wires into a field rather than a separate construction. A wire holding a bit must be pinned to $\{0, 1\}$, which one quadratic row expresses uniquely, and the integer must be tied to those bits:

$$b_i(b_i - 1) = 0 \quad (0 \leq i < w), \quad x = \sum_{i=0}^{w-1} 2^i b_i.$$

Every satisfying bit is zero or one because a field has no zero divisors. For a 64-bit range the sum lies in $[0, 2^{64} - 1] \subset [0, P - 1]$, so its canonical representative equals x as an integer. Conversely the ordinary binary expansion of any such x provides a witness. A field circuit cannot distinguish x from an external integer $x + P$; canonical input validation is therefore explicit. The compiled Q5 system takes the decomposition one bit wider than the bound, so that an out-of-range value is representable and the recovered comparator is what rejects it; the bit count alone never does the work.

S4.3 Q5: the range predicate

The predicate is built by threading (ℓ, e) through the recovered comparison cell from the most significant position down, initialised with ℓ false and e true, and reading the final “less”. At each position the scan selects the cell recovered for that bound bit; the scan itself contains no logic beyond that selection. Compiled at 65 bits against the bound 2^{64} , the system is 335 rows over 200 signals.

The scan is exact at every width where it can be enumerated completely. For each width w from one to eight, and each of the 2^w bounds, the predicate was rebuilt from the recovered cells and evaluated on all 2^w inputs against the integer test $x < b$. This is $\sum_{w=1}^8 4^w = 87,380$ cases, and all 87,380 agree. What is not claimed is compiled-circuit verification: the pinned Circom

toolchain is absent from the recorded environment, so that status is UNKNOWN rather than PASS, as Section S7 records.

S4.4 Q6: exclusion of one

The exclusion is compiled from the same comparison cell run at 254 bits against P , which forces the element to be canonical, together with the recovered inequality cell scanned against the constant one. The system is 2,193 rows over 1,223 signals. Valid witnesses are checked at $r = 0, 2, P - 1$. The forgery worth naming is the modular alias: an assignment claiming $r = 1$ while carrying the bit pattern of $P + 1$, which is the same field element written noncanonically. Injected directly into the compiled rows, it is rejected by 102 of them, every one a gate row of the recovered scan, so it is the recovered cells themselves that refuse it. Rejection is read from the rows, never from a witness generator declining to produce an assignment.

A shorter relation is available to the field directly, $(r - 1)s = 1$, which has a witness exactly when $r \neq 1$, since one may take $s = (r - 1)^{-1}$ while at $r = 1$ the equation would read $0 = 1$. It is one row over two signals. It is retained as an independent cross-check on the compiled system rather than as the answer, and the size difference between them is reported plainly in Section S7.

S5 Q7: a 64-bit factorisation

S5.1 The multiplier built from recovered cells

The two cells this question needs are the last two rows of the experiment of Section S4.1. The full adder is stated as the integer $a + b + c_{\text{in}}$, with its low bit and its high bit taken as the two outputs; the partial product is stated as the one-bit product uv . Enumerating the first over its eight input states and the second over its four, the method returns all three inputs essential for both adder outputs, the sum as XOR and the carry as MAJORITY, and the product as AND. That is the classical full adder, and neither name occurs in what the method was given.

The multiplier is then a shift-and-add schedule over those two cells and nothing else. For each factor bit j the schedule places one recovered partial product per column in range and one recovered full adder per column of the running total, threading the carry along the column and retaining the stage carry at the end. No product bit is discarded by construction, and the stage carries are carried out of the network as outputs so that the compiler can constrain them to zero; the absence of truncation is therefore an explicit constraint rather than a property of the schedule. Compiled at the full 64 bits, the system is 107,271 rows over 53,572 signals.

S5.2 The exhaustive ladder and the law it fixes

Verification is exhaustive wherever exhaustion is possible and adversarial where it is not.

At the Boolean level the multiplier was enumerated at widths one to five. For every pair (u, v) the complete $2w$ -bit product must equal uv and every stage carry must be zero: $4 + 16 + 64 + 256 + 1024$ pairs, with no failure.

At the constraint level the serialized rows themselves were swept. For widths four, five and six, every triple (u, v, n) over the width was evaluated against the rows and against the independent integer predicate $u \geq 2 \wedge v \geq 2 \wedge uv = n$, and the two were required to agree in both directions — a satisfying triple must satisfy every row, and a non-satisfying triple must break at least one. Table 2 records the sweep. There is no discrepancy in 299,008 triples.

Two exact laws are fitted over the emitted systems from width four to width sixty-four and confirmed at 128: a width- w multiplier has $7w^2 + 1$ gates and compiles to $26w^2 + 12w + 7$ rows.

Table 2: The Q7 constraint ladder. Every triple at each width, checked against the compiled rows and an independent integer predicate. *Valid* is the number of triples the predicate accepts; the remainder must be rejected by the rows. The row counts are the compiled systems at that width and agree with $26w^2 + 12w + 7$ exactly.

Width	Triples	Valid	Rows	Discrepancies
4	4,096	16	471	0
5	32,768	52	717	0
6	262,144	148	1,015	0
Total	299,008	216	—	0

At $w = 64$ the row law reads $26 \cdot 4096 + 768 + 7 = 107,271$, which is the released system, so the ladder’s widths and the answered width are the same construction rather than two.

At 64 bits enumeration would require 2^{192} triples and is not attempted. Instead the released system is attacked. It accepts genuine factorisations, including semiprimes whose product occupies all 64 bits, the largest accepted product being 18,446,743,979,220,271,189. It rejects an all-zero assignment, a trivial factor in either position, a wrong public product, and a pair of 34-bit factors whose product overflows 64 bits and would otherwise pass as its truncated residue. The forged assignment recorded in the evidence keeps a valid pair $(u, v) = (3, 5)$ while claiming $n = 0$, and four rows reject it.

S5.3 The direct system, retained as a cross-check

The field can state the same condition in 200 rows. Range-constrain public n and private u, v to 64 bits, enforce $uv = n$, and enforce $(u - 1)s = 1$ and $(v - 1)t = 1$. The system also explicitly enforces $u, v \geq 2$ using an inverse for the sum of each factor’s bits above position zero. That sum lies between 0 and 63, so its nonzero field value means the factor is at least two. These extra rows make the public-input promise robust.

Completeness follows by taking the binary expansions and inverses from every nontrivial integer factorisation. For soundness, range constraints give $0 \leq u, v < 2^{64}$, so $uv < 2^{128} < P$. Both uv and n are therefore canonical integers, and the field equality lifts to $uv = n$ over the integers. The inverse constraints exclude trivial factors. The circuit checks supplied factors; it does not find them.

Both routes are at width 64, so the comparison between them is like for like: 107,271 rows against 200, a factor of 536. The direct system borrows the field’s own multiplication and the recovered one borrows none of it, which is the whole of the difference. No efficiency claim is made for the recovered route; what it establishes is that the gates performing the arithmetic were obtained from behaviour rather than asserted.

S6 Q8: a 4096-bit factorisation

Put $B = 2^{64}$ and represent u, v, n by 64 limbs each: $u = \sum_i u_i B^i$, $v = \sum_i v_i B^i$, and $n = \sum_i n_i B^i$. Range-constrain every limb, including public n_i , to 64 bits. Introduce 4096 partial-product

signals and 129 carries, with rows

$$\begin{aligned} u_i v_j &= q_{ij} & (0 \leq i, j < 64), \\ \sum_{i+j=k} q_{ij} + c_k &= n_k + B c_{k+1} & (0 \leq k < 128), \\ c_0 &= c_{128} = 0, \end{aligned}$$

where $n_k = 0$ for $k \geq 64$. Constrain $c_1, \dots, c_{127}$ to 70 bits. For each factor take the sum of its 4095 bits other than global bit zero and require an inverse of that sum. Since $4095 < P$, this enforces that the factor is at least two.

Completeness. Every integer factorisation $n = uv < 2^{4096}$ with $u, v \geq 2$ has factors below 2^{4096} . Use their base- B limbs, exact partial products, and ordinary schoolbook carries. Inductively $c_k \leq 64(B-1)$: with at most 64 products, the column total is at most $64(B-1)^2 + 64(B-1) = 64B(B-1)$. Hence $c_{k+1} \leq 64(B-1) < 2^{70}$. The output fits 64 limbs, so all higher output limbs and the final carry are zero. Binary decompositions and the indicated inverses complete the witness.

Soundness. Every $u_i v_j < B^2 = 2^{128} < P$, so each partial-product field equation fixes the exact canonical product q_{ij} . The left side of any column is at most $64(B-1)^2 + (2^{70} - 1)$; the right side is at most $(B-1) + B(2^{70} - 1)$. Both are nonnegative and less than $2^{135} < P$. Thus every column equality is an integer equality, not merely a congruence. Multiplying column k by B^k and summing cancels the intermediate carries, giving $uv = n + B^{128} c_{128} - c_0 = n$. The zero upper outputs prevent overflow past the 64 public limbs, and the inverse constraints make both factors nontrivial. This proves both directions independently of witness generation.

Why the recovered route does not reach this question. The construction above is what answers Q8, and index deconvolution is not what produces it. The reason is measured rather than asserted, and it is worth separating two different costs.

The first is the gate count of a bit-level multiplier, which would be the obvious way to extend the Q7 route of Section S5 to this width. Its scaling is the exact law fitted there, so $w = 4096$ gives 436,256,775 rows against 25,725 here, a factor of about seventeen thousand. That is an argument about expense only.

The second is decisive. The method's reach depends on the decision diagram staying small during the recovery, and whether it does is a property of the function rather than of the implementation. The quantity to compare is the allocation, every node the recovery had to hold, and it must be compared with allocation rather than with the recovered function's own description, which is a different and much smaller thing. Built on the same engine, majority's allocation grows as $n^{3.96}$ across six sizes from $n = 31$ to $n = 151$, reaching 41,196,123 nodes at $n = 151$; that is why Section S3's recovery climbs so far, and the recovered function's own diagram there is only 5,776 nodes. The multiplier's allocation grows by a factor of 2.977 per additional bit, measured across eleven widths from two to twelve, where it already occupies 4,384,542 nodes and exceeds an eight-million-node budget at thirteen. The per-bit ratios settle quickly — 5.778, 4.721, 3.688, 3.347, 3.094, 3.019, 2.983, 2.973, 2.958, 2.953 — so the extrapolation rests on a rate that has stopped moving rather than on a trend still in transient. Extrapolating it to 4096 bits gives roughly 10^{1942} nodes, against about 10^{80} atoms in the observable universe. At no point in that ladder does the recovered description become available to compare, because the recovery does not finish.

This is not an encoding that has yet to be found. Exponential growth of such descriptions for integer multiplication is a known lower bound [17], and the measurements are that bound

appearing in our own engine. The limb construction escapes it by never entering Boolean logic at all: each limb product is a single field multiplication, so the blow-up has no opportunity to arise. Recording the boundary states which problems this method is the correct instrument for, which omitting the question would not.

S7 Verification record and reproducibility

S7.1 Recovered-cell arithmetic route

The arithmetic questions are answered a second time out of gates that index deconvolution returned. No cell is written down: each is stated as an integer relation, its complete behaviour is handed to the method, and the recovered gate is compiled. The evaluator calls the existing CausalBool gate semantics [13]; a separate lowering pass emits the frozen serialized rows $A(w)B(w) = C(w)$ used by the challenge evaluator and Circom exporter. Witness generation and constraint enforcement stay separate: the serialized rows remain the acceptance boundary.

The six cells, the connected inputs and canonical names the method returned for each, and the match class within which each name is a choice, are recorded in Table 1 of Section S4.1, together with the expansion obligation and the root identity that discharges all nine expansions in use. This subsection records the lowering and the evidence rather than repeating that account.

The lowering supports AND, OR, XOR, NOT, three-input MAJORITY and Boolean constants. For example, the rows for AND and XOR are

$$ab = z, \quad 2ab = a + b - z, \quad (\text{S1})$$

with explicit Booleanity rows for inputs, outputs and auxiliary signals. Repeated operands are combined as coefficients in the linear expression, so the same contract applies to fan-out and duplicate ports. The resulting systems are:

Question	Signals	Rows	Verified scope
Q5	200	335	$x < 2^{64}$ from the recovered comparator
Q6	1,223	2,193	254-bit canonical comparison and exclusion of one
Q7	53,572	107,271	complete 64-bit multiplier and lower-bound checks

For Q5 the decomposition is 65 bits, one wider than the bound, so values at or above 2^{64} are representable and only the recovered comparator excludes them; a 64-bit decomposition would make the predicate vacuously true. For Q6 the scalar is packed into 254 bits, constrained below P , and required to differ from one. For Q7 the multiplier retains every product bit and its stage carries; the public product is range-bound and both factors are required to be at least two.

Verification is exhaustive where enumeration is possible and adversarial where it is not. The Q5 sweep over every width to eight, every bound and every input is given in Section S4.3 and the Q7 constraint ladder in Section S5.2; neither records a discrepancy against its independent integer predicate.

The recovered route is larger than the direct one at every question, and no efficiency claim is made for it. At Q7 the comparison is like-for-like, both at width 64, and the recovered system is 536 times the size. What it establishes is that the gates performing the arithmetic were obtained from behaviour rather than asserted.

The local serialized-row verifier and Boolean tests pass. In the recorded environment the pinned Circom and snarkjs binaries are unavailable, so the external compiled-row and

witness status for this route is UNKNOWN rather than PASS. The evidence and exact commands are recorded in `evidence/causalbool_arithmetic/verification.json` and `plan/causalbool_arithmetic/LUNA_REPORT.md`.

The exporter writes only generic quadratic equalities and input declarations. All private and auxiliary assignments are supplied explicitly. When the pinned tools are available, compilation uses Circom 2.2.3, BN128, optimisation level zero, and disabled witness sanity checks; the R1CS remains enforced. Snarkjs 0.7.6 checks valid witnesses. A separate Python evaluator reads exported compiled rows, and malicious assignments are evaluated directly against those rows. Witness-generator refusal is never counted as soundness evidence.

The audit covers range boundaries, nonbinary bits, factor zero and one, incorrect public products, partial products, carry bits and endpoints, high limbs, overflow, and modular aliases. An explicit false factorisation $2((P + 7)/2) = 7 + P$ satisfies multiplication columns with unrestricted field carries; the released carry ranges reject it. Deliberately removing the relevant range rows makes this negative control pass, demonstrating why those bounds are required. Public-limb and factor-range omission controls are checked too.

S7.2 Specified validation inputs

The 13 Fourier vectors at each size are: the zero vector; the constant vector $2 - 3i$; four impulses of amplitude $1 - 2i$ at positions $0, 1, \lfloor N/3 \rfloor, N - 1$; three modes $x_j = \exp(2\pi i k j / N)$ for $k = 1, N/4, N - 1$; and four complex Gaussian vectors. For the latter, independent standard normal samples supply the real and imaginary parts, using the default NumPy random generator with seeds 101, 202, 303 and 404. The recorded environment uses NumPy 2.4.3 and Python 3.13. The normalised maximum error is

$$\varepsilon = \frac{\max_r |y_r - y_r^{\text{ref}}|}{\max(1, \max_r |y_r^{\text{ref}}|)},$$

where the reference is the unnormalised forward transform evaluated by an independent FFT. A finite error no larger than 10^{-10} is required. Complex arrays use double precision. Small-size checks additionally compare every basis vector through $N = 128$ against the direct exponential formula. The target runs impose a 900-second limit per vector and a 4 GiB memory ceiling. Their plaintext elapsed times are validation costs, not estimates of encrypted execution.

The valid arithmetic witnesses exercise the following inputs, with $B = 2^{64}$. In Q7 and Q8 the public value is the product of the listed factors.

Question	Valid inputs
Q5	$x = 0, 1, B - 1$
Q6	$r = 0, 2, P - 1$
Q7	$(u, v) = (2, 3), (3, 2), (65537, 65539), (2, (B - 2)/2)$
Q8	$(u, v) = (2^{2047} + 123, 2^{2047} + 321), (2, 2^{4094} + 1), (2^{4094} + 1, 2)$

Valid assignments are checked both before and after compilation. Invalid assignments are injected directly into the compiled rows, so failure of a witness-generation routine is not used as rejection evidence. Mutations exercise nonbinary bits, out-of-range values, wrong inverses, zero and unit factors, changed public products, partial products, carry bits, endpoint carries, high factor and public limbs, and overflow beyond 4096 bits. The 64-bit range interface also rejects noncanonical external integers $-1, P, P + 1$ before they can alias field values.

S7.3 Why the omitted-bound controls are informative

For the large-factor carry control, take $u = 2$, $v = (P + 7)/2$ and public $n = 7$. The ordinary product is $P + 7$. Write the factors and n in limbs, set $c_0 = 0$, and compute successive field carries by

$$c_{k+1} = B^{-1} \left(\sum_{i+j=k} u_i v_j + c_k - n_k \right) \pmod{P}.$$

All multiplication columns are then satisfied. Summing the weighted columns shows that $c_{128} = B^{-128}(uv - n) = 0$ in the field, so even zero endpoints cannot reject the false integer equality by themselves. The released system rejects these carries because of its 70-bit range equations. Deleting those range equations makes this assignment pass, isolating their role.

The small-factor control uses the same modular alias and isolates the 64-bit factor bound. A public-limb control replaces n_0 by $n_0 + B$ and n_1 by $n_1 - 1$, leaving the represented integer unchanged but violating the canonical limb range. A separate overflow control attempts to represent $2^{4095} \cdot 2$ with all public limbs zero. These controls check distinct ways in which a field assignment could violate the intended integer statement.

Checks behind the operation-network diagrams. The query diagram computes the sum and weighted contributions of $(3, 8, 19)$, then repeatedly divides the exact answer by 31 and checks recovery of all three entries. The eight-slot Fourier diagram is generated from three explicit butterfly matrices and the bit-reversal permutation. Their product is compared with the direct exponential matrix, and the emitted packed circuit is checked on all eight basis vectors against that same formula, with a maximum-error tolerance of 10^{-12} . The target-size cost bars and stage groups are read from the recorded search results. For the decimal multiplication illustration, the successive column totals are 21, 28, 10, 1, with output digits 1, 8, 0, 1 and outgoing carries 2, 2, 1, 0. The check verifies that their weighted digit sum equals $23 \cdot 47 = 1081$. These small examples explain the operations; the large-dimension numerical and compiled checks remain those reported above.

S7.4 Constraint counts

For Q8 the 192 factor and public limbs contribute $192 \cdot 65 = 12480$ rows: 64 bit equations and one reconstruction equation per limb. The 4096 partial products contribute 4096 rows. The 127 intermediate carries contribute $127 \cdot 71 = 9017$ rows: 70 bit equations and one reconstruction each. Add 128 column equations, two zero-endpoint equations and two factor lower-bound equations, for 25725 rows in total. Auxiliary signals are supplied explicitly. The signal count is $192 \cdot 65 + 4096 + 129 + 127 \cdot 70 + 2 = 25597$. For Q7, three 64-bit decompositions, one product and four inverse equations give $195 + 1 + 4 = 200$ rows. The 65-row range gadget and one-row exclusion gadget complete the four families.

S7.5 Reading the role ledger

A method is only as useful as the claims it actually carries, so the generated ledger below states which of three roles index deconvolution plays for each answer. It **derives** an answer when the object compiled into that answer is what the method returned. It **certifies** an answer when the answer is argued another way and the method recovers and verifies a decisive Boolean part of it exactly. It **bounds** an answer when it does not produce the answer at all and what it contributes is a measurement of why.

Generated results

N	Multiplies	Adds	Rotations	Depth	Cost (s)
32768	4757	4754	433	3	29.299516
65536	13313	13310	570	3	78.083040

For $N = 32768$, stages are $[7, 7, 1]$, baby sizes $[2048, 32, 256]$, and the dense baseline is 185.962068 seconds (6.347 times the selected cost). The 13 target cases have maximum normalised error 2×10^{-15} . For $N = 65536$, stages are $[8, 7, 1]$, baby sizes $[4096, 32, 256]$, and the dense baseline is 370.374740 seconds (4.743 times the selected cost). The 13 target cases have maximum normalised error 2.13×10^{-15} .

n	1	3	5	7	9	11	13	15
Gates	0	1	7	30	88	205	411	742

Direct arithmetic systems

System	Compiled rows	Signals
Q5	65	65
Q6	1	2
Q7	200	199
Q8	25725	25597

Recovered-cell arithmetic systems

System	Rows	Signals
Q5	335	200
Q6	2193	1223
Q7	107271	53572

Role ledger

Question	Role
Q1	CERTIFIES
Q2	DERIVES
Q3	CERTIFIES
Q4	CERTIFIES
Q5	DERIVES
Q6	DERIVES
Q7	DERIVES
Q8	BOUNDS

Recovered cells, as the method named them from stated integer relations: comparator bound 0 = AND, NIMPLIES; comparator bound 1 = REGULATORY_DNF, AND; difference bit 0 = OR; difference bit 1 = REGULATORY_DNF; full adder = XOR, MAJORITY; partial product = AND. All 9 expansions into the constraint gate set are verified by root identity. Majority is recovered at $n=151$ with every input essential and identity against an independent threshold, enumerating 0 states. Recovering it allocated 41,196,123 decision nodes, growing as $n^{3.96}$; the

recovered function’s own diagram is 5,776 nodes, growing as $n^{1.97}$, and equals $(n + 1)^2/4$ at every size in the ladder. The exact match class is KOFN and MAJORITY at $n=31$, so the reported name is the canonical priority order, not a unique identification. The Q7 constraint ladder is exhaustive at widths 4 (4,096), 5 (32,768), 6 (262,144): 299,008 triples with 0 discrepancies against an independent integer predicate. Recovery allocation grows as $n^{3.96}$ for majority, its description as $n^{1.97}$, and the multiplier’s allocation by a factor of 2.977 per bit, so a 4096-bit multiplier would need about 10^{1942} nodes against roughly 10^{80} atoms in the observable universe. The compiled audit records 66 passing checks. The challenge and affected Boolean regression suites collect 90 and 36 tests respectively; all pass. The modular enumeration counts for primes 3, 5, 7, 11, 13, 19 are 9, 25, 1, 1, 73, 1. Tables are generated from the recorded verification results; individual records retain the producing commands and source hashes. The supplementary index check reconstructs 2730 majority circuit states. The incorrect nine-input circuit disagrees with majority on 54 of 512 states, despite having the same functional inputs and the same number of active indices.

S7.6 Reproduction and scope of the evidence

Reproduction proceeds in four parts: regenerate the discrete constructions and compare their complete or symbolic outputs; emit and audit the four quadratic systems; execute both selected Fourier circuits and compare their results; and regenerate the tables and documents from the resulting records. The accompanying reproduction guide supplies executable commands and dependency pins separately from the mathematical exposition.

The original verification record contains 63 challenge tests and 525 affected Boolean regression tests. It also records 66 compiled arithmetic checks. Missing tools, skipped required tests and unresolved outcomes are failures of full verification. The evidence retains records for the three omitted questions, since those computations were run and their results stand; they are simply not reported as results of this method. The new index-deconvolution examples are a separate, bounded manuscript check, described in Section S3. The original numerical and compiled results are retained; rebuilding the documents does not rerun those experiments. Records retain the producing commands, source hashes, numerical tolerances, input sizes, seeds, elapsed times and observed memory. Document validation checks successful compilation, resolved references and page bounds. It does not replace the mathematical or computational checks.

What has not been measured. No complete 2025-input circuit has been materialised, no ciphertext Fourier timing has been taken, and no zero-knowledge proof has been generated. The arithmetic results establish properties of the compiled constraint systems and supplied witnesses. They are not a benchmark of proving time, proof size or a deployed cryptographic protocol.

S8 General capacities and developmental scope of index deconvolution

The index-deconvolution method is the Boolean-network component used in this work. Its conceptual lineage is the algorithmic information-dynamics and programmability framework developed for analysing how integrated systems respond to interventions, how generative rules expose their behaviour, and how such descriptions can support explanations and predictions [2, 3, 10, 11]. The method is strongly aligned with the Kolmogorov–Chaitin view of information as program size and with its modern formal treatment [4, 5, 6]. In parallel, its interest in how simple rules can generate rich behaviour is consonant with Wolfram’s Principle of Computational Equivalence, namely the proposed computational equivalence in sophistication of many processes

above a low threshold [7, 8]. We use that principle as a conceptual motivation, not as a theorem about the present Boolean examples. The perturbational and causal interpretation follows Zenil and collaborators’ model-based interventional calculus and causal deconvolution work [9, 10, 11]. The present method is a specialisation of these ideas to exact finite Boolean descriptions, and remains distinct from a computation of Kolmogorov complexity itself.

S8.1 The capacity map

For a synchronous Boolean network, the forward index-deconvolution representation maps a connectivity matrix and node rules to an ordered repertoire. The inverse method treats the network and its evolution as one integrated system, rather than as a collection of independently reduced tables. Given node sets $S_1, \dots, S_m$, it can ask for a pattern associated with their union $U = \bigcup_{r=1}^m S_r$ under a stated dynamical condition, such as an initial state, time window, intervention or output coordinate. For the single-output illustration used in the checked Boolean example, a coordinate i has an output-changing witness exactly when there is a state x such that

$$O_k[x] \neq O_k[x \oplus 2^i]. \quad (\text{S2})$$

Such a witness isolates a dependence under the stated condition; it does not cut a piece out of the integrated dynamics. Changing U changes the observation and conditioning map, and can therefore change the set of outputs that is obtained. The isolated pattern is consequently not asserted to be a set-theoretic subset, intersection, or independent sub-behaviour of the whole system. It is an exact observation of the integrated evolution under the specified node union and condition.

When the supplied description contains the complete transition map, its evolution can be iterated from every admissible state. A lossless pattern description can then account for all outputs and attractor cycles entailed by that description, including those seen through different unions of node sets. This attractor statement requires a transition description; the static single-step repertoire checked here supports exact output and perturbation claims but is not by itself a dynamical attractor analysis.

The main capacities and their exact boundaries are summarised below. The word “exact” always refers to the stated finite model and its supplied repertoire; it does not mean that an arbitrary hidden implementation is uniquely identifiable.

Figure 1 displays these operations in the same network–matrix–pattern style as the worked majority figures. It uses the five-input circuit already checked in this response: the complete repertoire is the input to the inverse, paired perturbations isolate an output condition, and the schema view gives a lossless description of that condition in the chosen index representation. The final panel closes the loop by replaying the integrated description and answering a deterministic query. The static example visualises one output condition; the general method applies the same logic to unions of node sets and, when a transition map is supplied, to evolution and attractors.

S8.2 Compression and the Kolmogorov perspective

The schema representation makes the algorithmic-information connection concrete. A schema fixes some coordinates and leaves others as don’t-cares. This is a fixed, exact code for the supplied finite behaviour, whereas Kolmogorov complexity is defined by the shortest program relative to a fixed universal description language [4, 5, 6]. The distinction is important when comparing the present decoder with the Block Decomposition Method (BDM), which uses algorithmic-probability/CTM estimates on local blocks and combines them into a global complexity estimate [12]. BDM is a general approximate complexity framework; the index-deconvolution schemas

Capacity	What the method does	Boundary of the claim
Exact representation	Represents the complete Boolean behaviour as an ordered index/output repertoire and can expand a compact description back to that repertoire.	Requires a deterministic, correctly ordered finite repertoire.
Integrated-system querying	Considers the whole network and asks for exact patterns under unions of selected node sets and stated dynamical conditions.	The result depends on the observation and condition; it is not an independent component of the global dynamics.
Perturbational isolation	Uses paired interventions to witness whether a coordinate changes a specified output under the supplied integrated behaviour.	It isolates conditional dependence, not every structural edge or a reduced truth table.
Generative identification	Matches an exact observed pattern to a declared model family, or retains a lossless decoder when no named description is available.	Different internal circuits or model names can realise the same integrated behaviour.
Lossless compression	Groups output-one indices into schemata using decimal anchors, decimal families and sumandos; schema expansion reproduces the on-set exactly.	The description is a computable representation and need not be shorter for every function.
Dynamic accounting	Iterates a complete transition description to enumerate the outputs and attractor cycles entailed by the supplied model, including node-union observations.	A static repertoire alone cannot establish attractors; partial or noisy trajectories require additional assumptions.
Prediction by replay	Evaluates a queried condition and checks the result by independent forward replay of the integrated description.	It is exact within the model/data domain, not an out-of-sample forecast for noisy or incomplete data.

Table 3: Capacities and boundaries of the index-deconvolution method. The method separates what is recoverable from a complete repertoire from what remains non-identifiable.

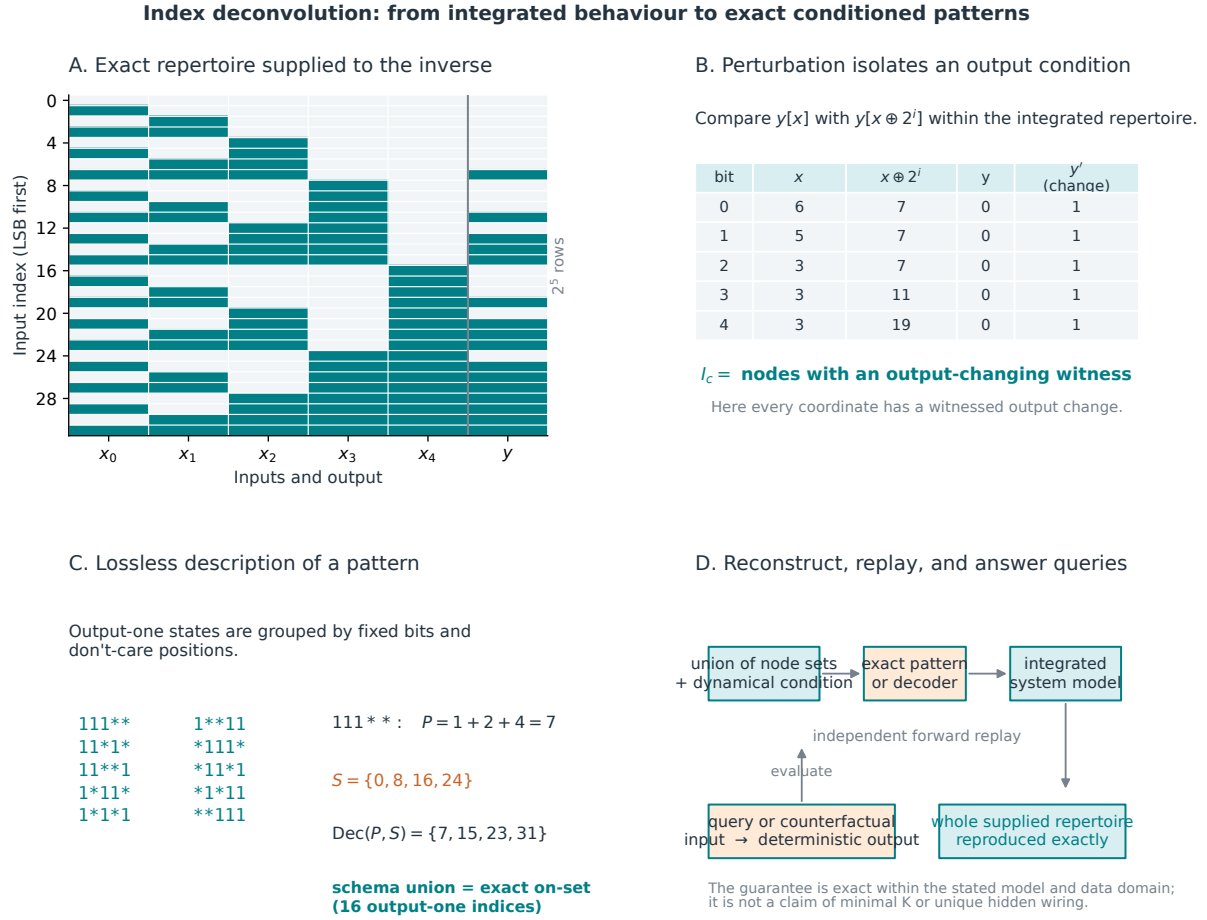


Figure 1: Capacities of index deconvolution. The actual five-input majority example is followed from its complete integrated repertoire through paired perturbations, isolation of an output condition, schema-based description and independent replay. The schema example has decimal anchor $P = 7$ and sumandos $S = \{0, 8, 16, 24\}$, whose exact expansion is $\{7, 15, 23, 31\}$. In the general case, unions of node sets and dynamical conditions select observations of the integrated evolution; they are not asserted to be set-theoretic subsets or intersections of a global behaviour. The diagram is a visual summary of finite exact operations, not a claim that the method computes minimal Kolmogorov complexity or identifies hidden wiring uniquely.

here are an exact finite-domain representation, not a BDM estimate and not a direct evaluation of K .

Its fixed coordinates determine a decimal anchor P ; the fillings of its own don't-care positions form the sumando family S . Across a decimal family L of anchors, the decoder expands the representation by

$$\text{Dec}(L, S) = \{\ell + s : \ell \in L, s \in S_\ell\}. \quad (\text{S3})$$

The expansion is checked against the original output-one index set, so this is lossless representation rather than a probabilistic approximation. When a function has repeated structure or overlapping regularities, the schema description can be substantially more compact than listing every active index. For functions such as XOR, however, the same representation can offer little or no compression. Compression is therefore a capacity to be measured for a specified function, not a property to be assumed in advance.

Kolmogorov complexity $K(x)$ is the length of the shortest halting program that produces x and is uncomputable in general [4, 5, 6]. A fixed index-set encoding therefore supplies a computable description length, and hence an upper-bound candidate relative to that encoding, rather than $K(x)$ itself. The method does not establish minimality across all programming languages, coding schemes or representations. Its algorithmic character comes from describing an individual deterministic object by a generative rule and testing whether that rule reproduces the object exactly. This is the sense in which the method is compression-oriented while retaining a rigorous distinction between an exact decoder and an estimated algorithmic-complexity value.

S8.3 Prediction, explanation and scale

Once a pattern has been isolated under a specified node union and dynamical condition, the integrated description can be replayed for a new admissible condition without consulting the original hidden implementation. Schema membership gives the same answer for an input in the represented domain, while a paired state $x \oplus 2^i$ supplies a direct witness for whether coordinate i can affect the observed output. In this limited but precise sense, the method supports prediction and explanation together: the predicted output is accompanied by the selected condition, the integrated generative description, and the exact replay check. This is the model-driven explanatory role that motivates the perturbation framework in the earlier work [2, 3, 10, 11].

The computational boundary is equally important. Given the full $2^n \times n$ repertoire, exhaustive paired checks for all columns take $O(n^2 2^n)$ time, but obtaining and storing that repertoire already costs an exponential number of states. Exact inversion is therefore practical for small networks and for larger systems only when an additional structural condition, such as bounded locality, makes the relevant observations effectively complete. Partial, noisy or non-recurrent observations require a different approximate problem and are not silently treated as exact deconvolution.

The method remains in development. The current implementation establishes finite exact reconstruction and lossless schema expansion for the checked Boolean cases; it does not yet provide a universal minimal description, representation-invariant complexity value, unique recovery of internal gate wiring, or a general out-of-sample predictor. Larger-network trajectories, partial observations, richer dynamical systems and principled comparisons of description lengths remain development directions. These qualifications are part of the method's scientific specification rather than caveats added after the result.

S9 What the majority ladder measures

The majority ladder reported for Q2 is the strongest evidence in this response, and it is also the place where a number can most easily be read as saying more than it does. This section separates

the quantities involved, gives the measured values, and states which of them is a property of majority and which is a property of the act of recovering it.

S9.1 Four counts, not one

The diagram manager is a shared store. It accumulates every function it has ever been asked to build and never discards one, because sharing is exactly what makes the representation canonical. A count taken from the manager is therefore a cumulative cost, not the size of any particular object. Four distinct quantities arise in one run at a given n :

Field	What it counts
composition nodes	nodes allocated while pushing the emitted MAJ3 circuit through the manager, one gate at a time; this includes the diagram of every intermediate wire
recovery nodes	nodes allocated by the recovery itself: the cofactor pass that tests each coordinate for essentiality, and the construction of every candidate gate whose identity is tested
specification nodes	nodes allocated when the threshold specification is built
recovered diagram nodes	nodes reachable from the recovered root, which is the size of the recovered function’s own description

Only the last is a property of majority. The first three are the price of obtaining and checking it.

S9.2 Measured values

n	Gates	Composition	Recovery	Specification	Description
15	742	3,955	726	0	64
21	2,935	15,270	1,728	0	121
31	14,190	72,655	4,918	0	256
41	43,720	222,290	10,658	0	441
51	105,025	531,925	19,698	0	676
63	245,086	1,237,799	35,958	0	1,024
101	1,623,175	—	—	—	2,601
151	8,116,950	—	—	—	5,776

Sizes to $n = 63$ are rebuilt on every certificate run. At $n = 101$ and $n = 151$ the totals are recorded and reproduced; the description size at those two sizes is measured directly and cheaply, because building the threshold function on its own costs milliseconds, and it is the recovered object’s size precisely because the recorded run established that the recovered root and the threshold specification are the same node.

The sum of the first three columns is the **decision nodes** total quoted in the main text, which is why that total is dominated by composition: at $n = 31$ the circuit accounts for 72,655 of 77,573 nodes, the recovery for 4,918, and the specification for none.

S9.3 Two growth rates, and why they differ

Fitted over $n = 31$ to $n = 151$, the allocation grows as $n^{3.96}$ and the description as $n^{1.97}$. The description obeys a closed form,

$$|\text{diagram}| = \frac{(n+1)^2}{4}, \quad (\text{S4})$$

which is verified at every size in the ladder rather than fitted, and which the certificate asserts on each run. The reason is a counting argument. Reading the coordinates in order, the only thing a threshold function must remember is how many inputs have been one so far, so a node is determined by a position and a running count, and there are $O(n^2)$ such pairs. This is the justification offered in the main text for why the method climbs to 151 inputs, and it is a statement about the 5,776-node object, not about the 41,196,123-node allocation.

The two exponents answer different questions. The description exponent says what it costs to *hold* majority at a given width. The allocation exponent says what it cost to *recover* it from one particular eight-million-gate circuit, and it would change if a different circuit computing the same function were supplied. Only the description exponent is a property of the function.

S9.4 What the independence of the specification amounts to

The check that the recovered object is majority compares the composed root against the threshold builder called with every coordinate and a bound of $\lfloor n/2 \rfloor + 1$. The canonical MAJORITY family in the gate builder is defined by that same call under the default strict tie policy, so naming the gate MAJORITY and matching the specification are one test, not two. The measurement makes this visible: `specification nodes` is zero at every size, because the specification was already present in the shared store.

What remains, and is not circular, is the content that matters. The circuit is produced by a recursive weight-merging construction that emits three-input majority gates and shares no code with the threshold builder, which is a dynamic program over coordinates. The identity therefore states that composing that circuit yields the counting function, over all 2^n inputs at once. It does not state that two independent specifications of majority were compared.

S9.5 The recovered name is a canonical choice, not a unique one

At $n = 31$ the recovery tests 162 candidates and two of them match the root exactly: MAJORITY with default parameters, and KOFN with $k = 16$. These are the same Boolean function, so both matches are correct and neither can be eliminated by any amount of behavioural evidence. The reported name is the earlier of the two in a fixed canonical priority order. The same two-member match class occurs at every size in the ladder. What the method identifies uniquely is the function; the name is a convention applied to it, and the full match class is recorded rather than discarded.

S9.6 Consequence for the comparison in Section S6

The multiplier ladder used to bound Q8 counts allocation while composing a circuit, with no recovery and no candidate gates. The like-for-like column for majority is therefore its allocation, and that is the column the comparison uses. Majority's allocation includes a recovery cost the multiplier's does not, which inflates the majority side by about six per cent at $n = 31$; this is far below the gap being described, and the conclusion is unaffected. The description sizes are reported beside the allocations so that the quadratic growth of one function and the per-bit multiplicative growth of the other are not read off columns that were produced by different procedures.

References

- [1] 0xPARC. *Beyond Primitive Computing: Puzzles and Connections*, Field Extension. <https://fielddextension.org/3b1b/>.
- [2] A. Hernández-Espinosa, H. Zenil, N. A. Kiani and J. Tegnér, *Estimations of Integrated Information Based on Algorithmic Complexity and Dynamic Querying*, arXiv:1904.10393 [q-bio.NC] (2019). <https://arxiv.org/abs/1904.10393>.
- [3] A. Hernández-Espinosa, H. Zenil, N. A. Kiani and J. Tegnér, “Estimations of Integrated Information Based on Algorithmic Complexity and Dynamic Querying,” in *Handbook of Unconventional Computing*, Vol. 1: Theory, ed. A. Adamatzky (World Scientific, 2021), Chap. 5, pp. 171–220. https://doi.org/10.1142/9789811235726_0005.
- [4] A. N. Kolmogorov, *Three approaches to the definition of the concept of quantity of information*, *Problems of Information Transmission* 1(1), 3–11 (1965); English reprint, *Three Approaches to the Quantitative Definition of Information*, *International Journal of Computer Mathematics* 2(1–4), 157–168 (1968). <https://doi.org/10.1080/00207166808803030>.
- [5] G. J. Chaitin, *On the length of programs for computing finite binary sequences*, *Journal of the ACM* 13(4), 547–569 (1966). <https://doi.org/10.1145/321356.321363>.
- [6] M. Li and P. M. B. Vitányi, *An Introduction to Kolmogorov Complexity and Its Applications*, 4th ed., Texts in Computer Science (Springer, Cham, 2019). <https://doi.org/10.1007/978-3-030-11298-1>.
- [7] S. Wolfram, *A New Kind of Science*, Chap. 12 (Wolfram Media, Champaign, IL, 2002). <https://www.wolframscience.com/nks/>.
- [8] S. Wolfram, “Notes for Chapter 12: The Principle of Computational Equivalence,” in *A New Kind of Science Online* (2002). <https://www.wolframscience.com/reference/notes/1128c/>.
- [9] H. Zenil, A. Schmidt and J. Tegnér, *Causality, Information, and Biological Computation: An Algorithmic Software Approach to Life, Disease, and the Immune System*, in *From Matter to Life: Information and Causality*, eds. S. I. Walker, P. C. W. Davies and G. F. R. Ellis (Cambridge University Press, 2017), pp. 244–280. <https://doi.org/10.1017/9781316584200.011>.
- [10] H. Zenil, N. A. Kiani, F. Marabita, Y. Deng, S. Elias, A. Schmidt, G. Ball and J. Tegnér, *An Algorithmic Information Calculus for Causal Discovery and Reprogramming Systems*, *iScience* 19, 1160–1172 (2019). <https://doi.org/10.1016/j.isci.2019.07.043>.
- [11] H. Zenil, N. A. Kiani, A. A. Zea and J. Tegnér, *Causal deconvolution by algorithmic generative models*, *Nature Machine Intelligence* 1, 58–66 (2019). <https://doi.org/10.1038/s42256-018-0005-0>.
- [12] H. Zenil, S. Hernández-Orozco, N. A. Kiani, F. Soler-Toscano, A. Rueda-Toicen and J. Tegnér, *A Decomposition Method for Global Evaluation of Shannon Entropy and Local Estimations of Algorithmic Complexity*, *Entropy* 20, 605 (2018). <https://doi.org/10.3390/e20080605>.
- [13] A. Hernández-Espinosa, *CausalBool*, source code accompanying this response, 2026.

- [14] G. Cohen, I. B. Damgård, Y. Ishai, J. Kölker, P. B. Miltersen, R. Raz, and R. D. Rothblum. *Efficient Multiparty Protocols via Log-Depth Threshold Formulae*, 2013. https://www.wisdom.weizmann.ac.il/~ranraz/publications/Pmajority_mpc-1.pdf.
- [15] J. H. Cheon, K. Han, and M. Hhan. *Faster Homomorphic Discrete Fourier Transforms and Improved FHE Bootstrapping*. Cryptology ePrint 2018/1073. <https://eprint.iacr.org/2018/1073>.
- [16] iden3. *Circom 2.2.3* and *snarkjs 0.7.6*. <https://docs.circom.io/>; <https://www.npmjs.com/package/snarkjs/v/0.7.6>.
- [17] R. E. Bryant. *On the Complexity of VLSI Implementations and Graph Representations of Boolean Functions with Application to Integer Multiplication*. IEEE Transactions on Computers 40(2):205–213, 1991.